Инфраструктура и измерения

Compose → readiness → воспроизводимость → время/RAM/диск/рост

Что нужно знать

- Порядок db healthy → loader success → pipeline success → app. Почему depends_on с успешным завершением отличается от простого запуска процесса.
- Named volume базы и bind mounts для CSV, модели и измерений; почему секреты и сырые данные исключены из Git и Docker build context.
- Методика: три повторения, среднее и отдельные результаты, одна изменяемая величина, характеристики ноутбука.
- Полный запуск 31.10 s относится к готовым CSV и образам; это не время первого скачивания и сборки.
- Индекс сравнивается на одной выборке 300 клубов; 25/50/100% эксперименты — отдельная агрегация крупнейшей таблицы.
- Память — максимальное наблюдаемое working set из docker stats, а не непрерывно измеренный истинный пик.
- Что нужно подготовить к защите: данные и образы заранее, рабочий ноутбук, сохранённые результаты, PDF и двухминутное резервное видео.

Что показать

1. Открыть docker-compose.yml и проследить зависимости и mounts без публикации паролей.
2. Показать README/MEASUREMENTS.md: машина, три прогона, диапазон условий и рост данных.
3. Объяснить отличие 31.10 s запуска от 10.872/0.0748 s запросов и от времени импорта одного CSV.

Короткая речь на английском

> *Docker Compose coordinates four core services. The database must be healthy, loading and processing must finish successfully, and then the application starts. Source files, the model and measurement results use explicit mounts. We measured timing, observed memory, disk footprint and growth in separate experiments, each repeated three times. Complete cached-source startup averaged 31.10 seconds on an Apple M3 Pro with 18 GiB RAM. The selective index lookup improved by about 145 times, but that result applies to that query and sample. Caches were retained, image builds were excluded and brief memory peaks may have been missed.*

Переход: “Taken together, the system is reproducible on the measured setup, while the data and model limitations remain explicit.”

Практические задания

1. По схеме объясни, что произойдёт, если loader завершится с ошибкой, а база уже healthy.
2. Сравни 25% и 100%: строк в 4 раза больше, загрузка примерно в 4.40 раза дольше; объясни, почему это не универсальный закон.
3. Подними проект по README на другом ноутбуке и запиши наблюдения. Не объявляй эту проверку выполненной до реального запуска.

Вопросы по направлению

What starts when you run Docker Compose?

Что запускает Compose?

The database becomes healthy, then the loader must finish successfully, then the processing pipeline must finish, and finally the application starts. The measurement service is on demand and the legacy interface uses an optional profile.

What does the 31.10-second measurement include?

Что входит в 31.10 секунды?

A mean over three complete starts with prepared source files and existing images: containers are recreated, sources reimported and the model retrained until the API is ready. Network acquisition and image builds are excluded and caches are not flushed.

Does the index make the whole application 145 times faster?

Всё приложение быстрее в 145 раз?

No. The measured gain is for 300 identical selective club lookups on the raw performance table, comparing index conditions. The frontend reads prepared outputs and a full GROUP BY has a different access pattern.

Is your memory number a guaranteed peak?

Это гарантированный пик RAM?

It is the largest observed Docker working-set sample during three runs, sampled at roughly two-second intervals. Brief peaks may be missed. We report the method rather than label it a continuous or hardware-independent maximum.

Why did four times as many rows not give exactly four times the query time?

Почему масштабирование не точно линейное?

Fixed costs, query plans, indexing, caches and hardware influence scaling. In this experiment four times the largest-table rows gave about 4.40 times the loading time and 3.35 times the aggregation time. That does not define a general law.

What remains before a group submission?

Что ещё нужно для сдачи?

Confirm actual team membership and each person’s contribution, run an independent second-laptop setup, resolve the source licence, rehearse the presentation and record a backup demo. Equal author email counts alone do not demonstrate real group participation.

What is the main result in one sentence?

Главный результат одним предложением?

The system is reproducible on the measured setup, while historical transfer spending alone has almost no linear relationship with next-season index change and the richer model adds only modest predictive value.

What is the most important limitation?

Главное ограничение?

The data are retrospective and unevenly covered, the index is a proxy, and associations are not causal effects. Source licence and independent team/laptop validation also remain explicit unresolved requirements.

How do goals for, goals against and goal difference work in the comparison?

Как в сравнении рассчитываются забитые, пропущенные голы и разница?

GF and GA are season totals rebuilt from completed home/away match scores. GD is GF − GA. They come from the optional football-data.co.uk match enrichment, not from the Champagne Index player totals.

How are attacking and defensive ranks calculated?

Как рассчитываются атакующий и защитный рейтинги?

Within one league and season, attack rank orders goals scored per match from highest to lowest; defense rank orders goals conceded per match from lowest to highest. SQL RANK gives tied rates the same rank and leaves a gap after ties. They describe scores, not tactics or chance quality.

Do we have xG or can we estimate it?

Есть ли в данных xG или можем ли мы его оценить?

No xG field is present in the loaded match schema or the checked source tables. Do not estimate xG from final scores. The comparison reports actual goals and per-match ranks only.

What is the difference between the Champagne Index and attack rank?

Чем атакующий рейтинг отличается от Champagne Index?

The Champagne Index is recorded player goals plus assists divided by a Transfermarkt matches proxy for a club-season. Attack rank uses actual completed league scorelines, sums team goals and ranks goals per match within the league-season. Their sources, coverage and units differ.

How did you check the goalkeeper defensive totals?

Как вы проверили вратарские показатели по пропущенным?

We compared Transfermarkt goalkeeper goals-conceded totals with the independent match-result totals on 388 adequately covered top-five club-seasons. They match exactly in 366 (94.3%). Clean sheets match only 269 (69.3%), so the scoreline source remains the basis of league ranks.

What does the defensive-spend analysis show?

Что показывает анализ расходов на защитников?

The Pearson correlation between defender transfer fees and next-season change in the goalkeeper-source conceded-goal rate is r=0.004047, p=0.426997, n=38,531. The sample does not support a useful linear association. It is descriptive, not causal.

Why keep the player-source defensive values separate from scoreboard goals?

Почему defensive-показатели по игрокам отделены от итогов матчей?

The sources use different records and denominators. The goalkeeper conceded total agrees with scorelines in 94.3% of the adequately covered sample, while clean-sheet totals agree in 69.3%. The UI therefore uses completed match scores for GF, GA, goal difference and ranks; the player-source totals are a data-quality cross-check.

Какие фразы не использовать

- 31 секунда включает скачивание 353 MiB и сборку образов.
- docker stats гарантирует истинный максимальный расход памяти.
- Одинаковое число коммитов подтверждает одинаковый личный вклад.

> Роль — учебная ответственность. Не заявляй личное авторство или самостоятельную проверку, если этого не было. Покажи ровно те изменения и команды, которые выполнял(а) лично.